Types of Load Balancers

1. In the AWS Management Console, launch two EC2 instances running a simple web server (use Amazon Linux 2 and install httpd or nginx), and verify both are accessible via their public IPs.

   ANS :

   1. Log in to the **AWS Management Console.**
   2. Open the **EC2 Dashboard**.
   3. Click **Launch Instance.**
   4. Configure the first instance:
      - Name: **WebServer-1**
      - AMI: **Amazon Linux 2**
      - Instance Type: **t2.micro**
      - Security Group: Allow **HTTP (80)** and **SSH (22).**
   5. Launch a second instance with the same configuration:
      - Name: **WebServer-2**
   6. Connect to each instance using SSH.
   7. Install Apache Web Server:
      sudo yum update -y
      sudo yum install httpd -y
      sudo systemctl start httpd
      sudo systemctl enable httpd
   8. Create different web pages on each server.
      WebServer-1
      echo "<h1>Welcome from Web Server 1</h1>" | sudo tee /var/www/html/index.html
      WebServer-2
      echo "<h1>Welcome from Web Server 2</h1>" | sudo tee /var/www/html/index.html
   9. Open each instance's **Public IP** in a browser and verify that both web pages are accessible.

2. Set up an Application Load Balancer (ALB) in AWS that distributes incoming HTTP traffic across your two EC2 instances, and test by accessing the ALB's DNS name in your browser to see traffic served from both servers.

   ANS :

   1. Open **EC2 → Load Balancers.**
   2. Click **Create Load Balancer.**

3. Select **Application Load Balancer**.
4. Configure:
    - Name: **MyALB**
    - Scheme: **Internet-facing**
    - Listener: **HTTP (Port 80)**
5. Select your **VPC** and **Public Subnets.**
6. Create a **Target Group**.
7. Register **WebServer-1** and **WebServer-2** as targets.
8. Create the Load Balancer.

➢ After the ALB is active, open its **DNS Name** in a browser and refresh the page multiple times. Requests should be distributed between both web servers.

3. Change the health check settings of your ALB to use a custom path (e.g., /health) and verify that the load balancer only routes traffic to healthy instances.

Hint: Create a /health endpoint on your web server that returns a 200 OK response.

ANS :

✓ **Configure the Target Group**

1. Open **Target Groups.**
2. Select your target group.
3. Click **Health Checks.**
4. Edit the settings:
    - Protocol: HTTP
    - Path: **/health**
5. Save the changes.
   Wait a few minutes until both instances show **Healthy.**

4. Simulate a server failure by stopping one of your EC2 instances and observe how the ALB handles requests — describe what happens and how this demonstrates high availability.

ANS :

1. Stop **WebServer-2** from the EC2 Console.
2. Wait until the target status changes to **Unhealthy.**
3. Open the ALB DNS name again.

- **Observation**

➢ The Application Load Balancer automatically detects that **WebServer-2** is unhealthy and stops sending requests to it. All incoming traffic is redirected to **WebServer-1**, ensuring the website remains available. This demonstrates **High Availability** because users can continue accessing the application even when one server fails.

5. Compare Application Load Balancer, Network Load Balancer, and Classic Load Balancer in a table with at least 3 points each, focusing on supported protocols, use cases, and features.

ANS :

Feature	**Application Load Balancer (ALB)**	**Network Load Balancer (NLB)**	**Classic Load Balancer (CLB)**
Supported Protocols	HTTP, HTTPS, HTTP/2, WebSocket	TCP, UDP, TLS	HTTP, HTTPS, TCP
OSI Layer	Layer 7 (Application Layer)	Layer 4 (Transport Layer)	Layer 4 and Layer 7
Best Use Case	Web applications, APIs, microservices	High-performance applications requiring very low latency	Legacy applications using the older AWS architecture
Routing Features	Path-based and host-based routing	IP address and port-based routing	Basic load balancing only
Performance	High performance for web traffic	Very high performance and millions of requests per second	Moderate performance
Example	E-commerce websites, Zomato, Flipkart	Online gaming, financial systems, VoIP	Older web applications